SRI SIVASUBRAMANIYA NADAR COLLEGE OF ENGINEERING, KALAVAKKAM
(An Autonomous Institution, Affiliated to Anna University, Chennai)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
III Year CSE (V Semester)
Academic Year 2026-27
UCS3513 SYSTEM DESIGN LABORATORY

Lab Exercise 6 - Design and Implement an API Rate Limiter using Token Bucket

Objective

To implement a Rate Limiter using the Token Bucket algorithm in a Spring Boot application, use Redis to maintain rate-limit information, and analyze request control under normal and burst traffic.

Problem Statement

A web application may receive a large number of API requests from users, which can overload backend services. Design and implement a Rate Limiter to restrict the number of requests from each client.

Use the Token Bucket algorithm to allow or reject requests based on bucket capacity and refill rate. Store the token information in Redis so that the rate limit can be maintained across application instances.

Tasks

- Develop a REST API using Spring Boot.
- Configure Redis with the Spring Boot application.
- Identify clients using a client ID or IP address.
- Implement the Token Bucket algorithm with configurable bucket capacity and refill rate.
- Store the token count and last refill time in Redis.
- Allow requests when tokens are available and return HTTP 429 – Too Many Requests when the limit is exceeded.
- Generate normal and burst requests and observe the rate-limiting behaviour.
- Test concurrent requests and use Redis Lua scripting for atomic token updates.

Analysis

1. Explain the role of:
    - Token Bucket

    - Bucket Capacity
    - Refill Rate
    - Redis
2. Explain how the Token Bucket algorithm allows or rejects requests.
3. Explain why Redis is required when multiple application instances are used.
4. Compare Fixed Window, Sliding Window Log, Sliding Window Counter, and Token Bucket rate-limiting algorithms.
5. Analyze the behaviour of the system during normal and burst traffic.

Expected Learning Outcomes

After completing this experiment, students will be able to:

- Implement API Rate Limiting using the Token Bucket algorithm.
- Configure bucket capacity and refill rate.
- Use Redis to maintain centralized rate-limit information.
- Handle normal and burst API traffic.
- Understand rate limiting in distributed web applications.